CourseSimplified

Assignment 06: Final Project Report

San Jose State University

Department of Computer Science

CS 151: Object-Oriented Design

Professor Robert Nicholson

May 11, 2026

Group: Objectively_Outstanding_ Peoples

Submitted by:

- Delis Santiago
- Daniel Chin
- Andrew Dover
- Jan David Ella

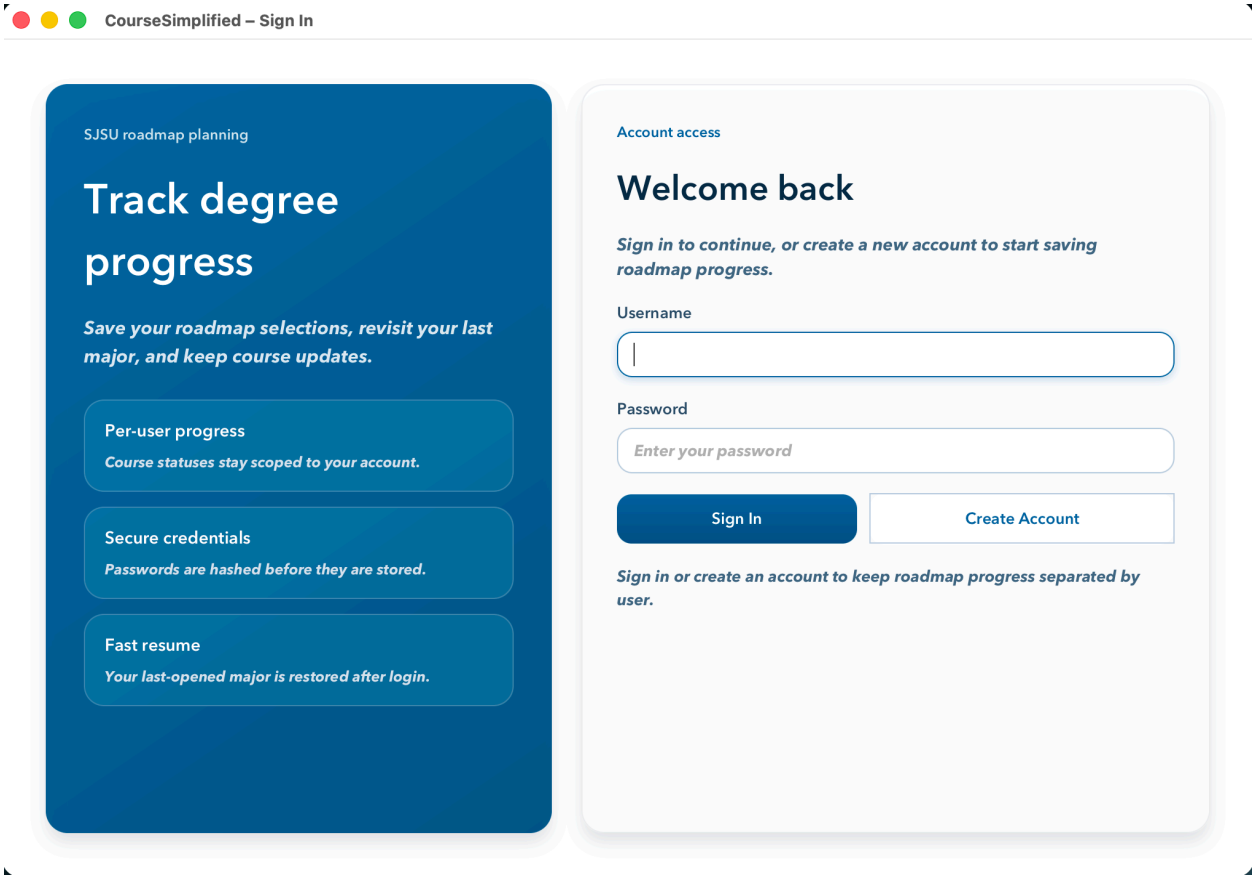
1. Description of the Project

CourseSimplified is an educational application which uses Java programming language and JavaFX framework to create its educational platform. The system was developed to assist students in tracking their progress through their chosen degree programs at San Jose State University. The system extracts course and degree information through an external API which it transforms into a directed graph that displays the prerequisite relationships between courses.

Through the application users have the ability to create an account and sign in to their account while choosing their major and accessing the corresponding roadmap and prerequisite tree. The user can also update course statuses as Remaining, In Progress, or Completed and monitor progress for the selected degree. The application stores user progress information between different user sessions, while it also keeps track of the last major field of study that each user selected.

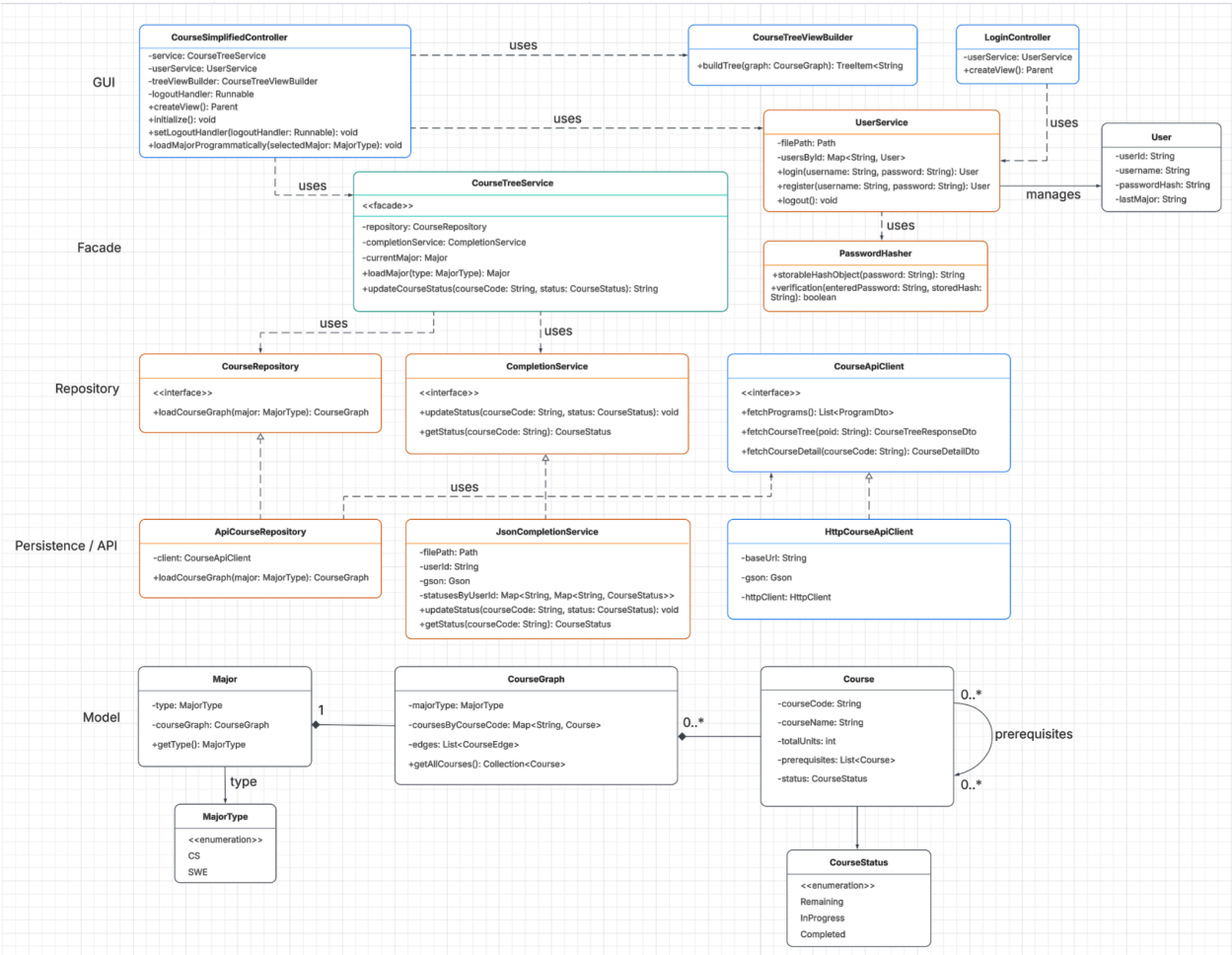
The design structure of CourseSimplified consists of multiple layers including an API client layer and a repository layer and a service layer and a JavaFX GUI layer. The application provides an efficient degree planning solution by displaying essential course requirements together with their prerequisite connections in an interactive and understandable format.

Figure 1: Screenshot of the CourseSimplified login interface.



2. Class Diagrams

Figure 2: UML Class Diagrams for CourseSimplified



Link to the Diagram: Lucidchart document

3. Design Principles

CourseSimplified implements multiple object-oriented design principles that enhance system maintenance and system understanding and system development. The project uses separation of concerns to distribute its functions between different operational layers. API client classes retrieve external data, repository classes construct roadmap data structures, service classes manage application logic such as roadmap loading, authentication, and persistence, and JavaFX GUI classes handle user interaction and presentation. The system design minimizes component dependencies while enabling each class to perform its dedicated operational function.

The project demonstrates encapsulation because it protects its internal state and implementation details through its design, which allows only the owning classes to access these elements. The system uses completion service classes to store course status information while UserService handles user account data and PasswordHasher manages password hashing functions. The program components interact with these system parts through established interface methods which prevent them from accessing internal system data.

The project uses abstraction as its third fundamental principle. The interfaces CourseRepository and CompletionService and CourseApiClient define all operational requirements needed to function at higher system levels without disclosing their internal workings. The system design enables operations to function through abstract interfaces instead of needing actual specific class implementations. The service layer can interact with any class that implements the needed repository and persistence and API functionality.

The project also uses decomposition into classes and relationships to model the problem domain clearly. Classes such as Major, CourseGraph, Course, MajorType, CourseStatus, and User represent core concepts, while service and repository classes manage how those concepts are loaded, updated, stored, and displayed. Relationships among these classes, especially prerequisite links between courses and user-specific progress tracking, help the design reflect the actual structure of academic planning.

The demonstration of inheritance and polymorphism through interface-based design shows its practical application. The classes ApiCourseRepository and JsonCompletionService and HttpCourseApiClient implement shared interfaces through their concrete operational functions. The system supports polymorphism because users can exchange one implementation for another without needing to modify code that operates at higher levels. The design achieves modularity and comprehensibility through these principles which match the project objectives.

4. Design Pattern(s)

CourseSimplified primarily uses the Facade pattern through the CourseTreeService class. This class provides a simplified interface for the rest of the application by handling roadmap loading, course status updates, prerequisite validation, and progress retrieval in one place. Instead of requiring the JavaFX controller to directly coordinate repository access and persistence logic, those layers interact with CourseTreeService. This reduces coupling and keeps business logic separate from the user-interface layer.

The project implements the Repository pattern through CourseRepository and ApiCourseRepository which serves as its implementation. The pattern enables service layer access to course data through an abstraction which protects high-level components from depending on specific software components. The design maintains its modularity through the interface-based approach which CompletionService and CourseApiClient use for their functions.

The application structure benefits from these patterns because they establish clear responsibilities which make maintenance work easier and future development work simpler.

5. Testing

CourseSimplified was tested using both automated unit tests and manual scripted tests. The automated tests focus on service logic and JSON persistence, while the manual tests verify the full workflow in the JavaFX GUI.

Automated Test Cases

Automated tests are located in src/test/java and are executed with mvn test.

1. **Load CS roadmap** (loadsComputerScienceRoadmap): Verifies that the Computer Science roadmap loads with the correct major name, total courses, and root course.

2. **Load SWE map** (loadsSoftwareEngineeringRoadmap): Indicates that the Software Engineering roadmap is being loaded in the correct manner.

3. **Update course to completed** (updatesValidCourseToCompleted): Checks that a valid course can be marked as completed and returns the correct message.

4. **Update course to In Progress** (updatesValidCourseToInProgress): Checks that a valid course can be marked as in progress without being counted as completed.

5. **Revert course to Remaining** (revertsCourseToRemaining): Verifies that a course can be reset to remaining status.

6. **Reject invalid course ID** (rejectsInvalidCourseIdsWithoutChangingProgress): Confirms that invalid course IDs are rejected and do not affect progress counts.
7. **Reject completion with incomplete prerequisites** (rejectsCompletionWhenPrerequisitesAreIncomplete): Verifies that a course cannot be marked as completed if the prerequisites for the course are not completed yet.
8. **Reject in-progress with incomplete prerequisites** (rejectsInProgressWhenPrerequisitesAreIncomplete): Verifies that a course cannot be marked as In Progress if the prerequisites for the course are not completed yet.
9. **Progress calculation** (progressCountsOnlyCompletedCourses): Verifies that completed, in-progress, remaining, and total counts are calculated correctly.

10. **Prerequisites eligibility logic** (eligibilityRequiresPrerequisitesToBeCompleted): Mandatory prerequisites must be ticked before the student is eligible for an advanced course.

11. **Legacy JSON compatibility** (loadsLegacyCompletedJsonWithoutBreakingExistingData): Verifies that the program can still load the older completed-only JSON format.

12. **Three-state JSON persistence** (persistsThreeStateStatusesAsCourseStatusMap): Verify that the current JSON format stores complete and in-progress statuses correctly.

Manual Scripted Test Cases

1. **Register account:** Create a new account and verify that the application accepts valid credentials and opens the planner.

2. **Login:** Sign in with an existing account and verify that the planner loads successfully.

3. **Invalid login handling:** Attempt to log in with an incorrect password or unknown username and verify that an error message is displayed.

4. **Logout:** Log out from the planner and verify that the application returns to the login screen.

5. **User-specific persistence:** Save progress under one account, sign out, sign in again, and verify that the same user's saved data is restored.

6. **Launch GUI:** Run mvn javafx:run and verify that the JavaFX window opens correctly.

7. **Load a major roadmap:** Select either Computer Science or Software Engineering and verify that the correct roadmap tree is displayed.

8. **Update course status:** Mark a valid course as Completed, In Progress, and Remaining, and verify that the tree and progress summary update correctly.

9. **Error handling:** Attempt to load a roadmap without selecting a major or enter an invalid course ID and verify that clear error messages are displayed.

10. **Prerequisite validation:** Attempt to mark a course as In Progress or Completed before its prerequisite is finished, and verify that the application blocks the update and displays an error message.

6. Design Process

The design of CourseSimplified started from its original conceptual model and developed into its current layered architectural framework. The project started with our team concentrating on academic planning domain modeling through the development of Course, Major, and prerequisite relationship classes. The development process showed that the system required better boundaries between its user interface, core business functions, data access operations, and data storage components.

The implementation of separate service and repository layers represents a fundamental design modification. The system divided API access, roadmap construction, course-status persistence, and presentation functions into separate classes which used CourseTreeService as the central interface for GUI access. The system gained better modularity through this modification which also decreased coupling while making maintenance tasks simpler.

Our team created three GUI classes which included CourseSimplifiedController and CourseTreeViewBuilder and LoginController while we kept the main application logic within the service layer. Our team developed the project into a multi-user system by implementing account registration and login functions with hashed password storage and user progress tracking features.

The status-tracking system experienced ongoing development throughout its operational history. The project updated its status tracking system from a basic completed/not completed status to three different course status categories of Remaining, In Progress and Completed. The final version introduced prerequisite validation as a new feature which prevented users from marking courses as In Progress or Completed until they had finished all necessary prerequisite courses.

The UML diagram underwent multiple revisions during the project to achieve better alignment with the completed work. Our team discovered that object-oriented design requires multiple design iterations to reach its final form. The team discovered three essential design principles which include separation of concerns and reliance on abstractions and design improvement through requirements evolution and system complexity growth.

7. Build Instructions

The Maven-based Java project CourseSimplified includes its source code organized into multiple packages, which contain GUI components, service-layer functionality, repository-layer components, API-access modules, model classes, and testing frameworks.

1. **Directory Structure**
    - **src/main/java/coursesimplified/gui:** Contains the JavaFX GUI classes, including the application entry point, controller, and tree view builder.
    - **src/main/java/coursesimplified/service:** Contains the service-layer classes for roadmap loading, course status updates, and persistence logic.
    - **src/main/java/coursesimplified/repository:** Contains repository classes responsible for loading course graph data.
    - **src/main/java/coursesimplified/api:** Contains API client classes used to retrieve course and program information from the external API.
    - **src/main/java/coursesimplified/model:** Contains the core domain model classes such as Course, Major, CourseGraph, MajorType, and CourseStatus.
    - **src/test/java:** Contains automated JUnit test cases.
    - **src/main/resources:** Contains JavaFX styling resources such as the application CSS file.

2. **Build Requirements**
    - Java 21
    - Maven 3.x

- Internet Connection for loading course data from the external API

3. **Build and Run Instructions**
   - Open your terminal in the project root directory.
   - Compile the project: mvn clean compile
   - Run automated tests: mvn test

4. **Run the JavaFX GUI**
   - Run: mvn javafx:run
   - The application opens to the login/register screen first, then loads the planner after sign-in.

5. **Notes for the Grader**
   - The JavaFX GUI is the primary submission interface.
   - The CLI was preserve from earlier development stage and still functions.
   - Course progress is saved locally in completed.json.
   - The application loads course roadmap data from API at runtime, so internet access is required for roadmap loading.

8. Conclusion

CourseSimplified demonstrates how object-oriented design principles, layered architecture, and appropriate design patterns can be applied to create a practical academic planning tool. Our team developed a functional system through the creation of a JavaFX graphical user interface and its associated service repository and model components. The project provided us with a better understanding of object-oriented analysis because we learned how iterative development improves software structure and user experience.